

# Taller Semana 8 — Estándares y Métricas de Calidad

---

## Pruebas Unitarias, Integración y Aceptación en Java

**Profesor:** Cesar Alfonso

**Programa:** Especialización en Desarrollo de Software

**Institución:** UNIMINUTO / Especialización en Desarrollo del Software

---



### 1. Objetivo del Taller

El estudiante implementará aseguramiento de calidad mediante:

- Pruebas **unitarias** con mocks.
- Pruebas **de integración** entre componentes reales.
- Pruebas **de aceptación** en estilo BDD (Given–When–Then).

El taller se desarrolla sobre una aplicación Java sencilla de gestión de usuarios.

---



### 2. Estructura del Proyecto

```
quality-testing-example/  
├─ pom.xml  
└─ src  
    ├─ main/java/com/example/quality/  
    │   ├─ model/User.java  
    │   ├─ repository/UserRepository.java  
    │   ├─ repository/InMemoryUserRepository.java  
    │   └─ service/UserService.java  
    └─ test/java/com/example/quality/  
        ├─ unit/UserServiceUnitTest.java  
        ├─ integration/UserServiceIntegrationTest.java  
        └─ acceptance/UserServiceAcceptanceTest.java
```



### 3. Dependencias (pom.xml)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
                              http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  
  <groupId>com.example</groupId>
```

```
<artifactId>quality-testing-example</artifactId>
<version>1.0.0</version>

<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <junit.jupiter.version>5.10.0</junit.jupiter.version>
  <mockito.version>5.11.0</mockito.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.jupiter.version}</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>${mockito.version}</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>${mockito.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
</project>
```

---

## 4. Código de la Aplicación

### 4.1 User.java

```
package com.example.quality.model;
```

```
public class User {

    private Long id;
    private String name;
    private String email;
    private boolean active;

    public User(Long id, String name, String email, boolean active) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.active = active;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public boolean isActive() { return active; }
    public void deactivate() { this.active = false; }
}
```

---

## 4.2 UserRepository.java

```
package com.example.quality.repository;

import com.example.quality.model.User;
import java.util.List;
import java.util.Optional;

public interface UserRepository {

    User save(User user);
    Optional<User> findById(Long id);
    Optional<User> findByEmail(String email);
    List<User> findAll();
    void deleteById(Long id);
}
```

---

## 4.3 InMemoryUserRepository.java

```
package com.example.quality.repository;

import com.example.quality.model.User;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
```

```
public class InMemoryUserRepository implements UserRepository {

    private final Map<Long, User> storage = new ConcurrentHashMap<>();

    @Override
    public User save(User user) {
        storage.put(user.getId(), user);
        return user;
    }

    @Override
    public Optional<User> findById(Long id) {
        return Optional.ofNullable(storage.get(id));
    }

    @Override
    public Optional<User> findByEmail(String email) {
        return storage.values().stream()
            .filter(u -> u.getEmail().equalsIgnoreCase(email))
            .findFirst();
    }

    @Override
    public List<User> findAll() {
        return new ArrayList<>(storage.values());
    }

    @Override
    public void deleteById(Long id) {
        storage.remove(id);
    }
}
```

---

#### 4.4 UserService.java

```
package com.example.quality.service;

import com.example.quality.model.User;
import com.example.quality.repository.UserRepository;
import java.util.List;

public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User createUser(Long id, String name, String email) {
```

```
        if (name == null || name.length() < 3)
            throw new IllegalArgumentException("El nombre debe tener al menos 3
caracteres");

        userRepository.findByEmail(email).ifPresent(u -> {
            throw new IllegalStateException("Ya existe un usuario con ese email");
        });

        User user = new User(id, name, email, true);
        return userRepository.save(user);
    }

    public User updateUserName(Long id, String newName) {
        if (newName == null || newName.length() < 3)
            throw new IllegalArgumentException("El nombre debe tener al menos 3
caracteres");

        User user = userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Usuario no
encontrado"));

        user.setName(newName);
        return userRepository.save(user);
    }

    public User deactivateUser(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Usuario no
encontrado"));

        user.deactivate();
        return userRepository.save(user);
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }
}
```



## 5. Pruebas Unitarias (Mockito)

### Propósito

Validar la lógica del servicio sin depender del repositorio real.

### Ejecución

```
mvn test
```

## UserServiceUnitTest.java

```
package com.example.quality.unit;

import com.example.quality.model.User;
import com.example.quality.repository.UserRepository;
import com.example.quality.service.UserService;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.InjectMocks;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class UserServiceUnitTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    @DisplayName("Crear usuario válido debe llamar a save()")
    void createValidUser() {
        when(userRepository.findByEmail("cesar@example.com"))
            .thenReturn(Optional.empty());

        when(userRepository.save(any(User.class)))
            .thenAnswer(inv -> inv.getArgument(0));

        User u = userService.createUser(1L, "Cesar", "cesar@example.com");

        assertEquals("Cesar", u.getName());
        verify(userRepository, times(1)).save(any(User.class));
    }

    @Test
    @DisplayName("Nombre corto debe lanzar excepción")
    void shortNameThrows() {
        assertThrows(IllegalArgumentException.class,
            () -> userService.createUser(2L, "Ce", "x@example.com"));

        verify(userRepository, never()).save(any());
    }
}
```

```
@Test
@DisplayName("Email duplicado debe lanzar excepción")
void duplicatedEmailThrows() {
    when(userRepository.findByEmail("dup@example.com"))
        .thenReturn(Optional.of(new User(99L, "Otro", "dup@example.com",
true)));

    assertThrows(IllegalStateException.class,
        () -> userService.createUser(3L, "Juan", "dup@example.com"));

    verify(userRepository, never()).save(any());
}
}
```

## 6. Pruebas de Integración

### Propósito

Validar interacción real entre servicio y repositorio.

### UserServiceIntegrationTest.java

```
package com.example.quality.integration;

import com.example.quality.model.User;
import com.example.quality.repository.InMemoryUserRepository;
import com.example.quality.service.UserService;
import org.junit.jupiter.api.*;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

public class UserServiceIntegrationTest {

    private UserService userService;

    @BeforeEach
    void setup() {
        userService = new UserService(new InMemoryUserRepository());
    }

    @Test
    @DisplayName("Crear usuario y obtener lista debe incluirlo")
    void createAndList() {
        userService.createUser(1L, "Cesar", "cesar@example.com");

        List<User> users = userService.getAllUsers();
```

```
        assertEquals(1, users.size());
        assertEquals("Cesar", users.get(0).getName());
    }

    @Test
    @DisplayName("Desactivar usuario debe reflejarse en repositorio")
    void deactivateUser() {
        userService.createUser(2L, "Ana", "ana@example.com");

        User u = userService.deactivateUser(2L);

        assertFalse(u.isActive());
    }
}
```

---

## ✓ 7. Pruebas de Aceptación (BDD)

### Propósito

Validar criterios de usuario en estilo Given–When–Then.

---

#### UserServiceAcceptanceTest.java

```
package com.example.quality.acceptance;

import com.example.quality.model.User;
import com.example.quality.repository.InMemoryUserRepository;
import com.example.quality.service.UserService;
import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.*;

public class UserServiceAcceptanceTest {

    private UserService userService;

    @BeforeEach
    void setup() {
        userService = new UserService(new InMemoryUserRepository());
    }

    @Test
    @DisplayName("Given no user exists, When creating one, Then it is active")
    void createUserBDD() {
        User u = userService.createUser(1L, "UsuarioNuevo", "user@example.com");

        assertTrue(u.isActive());
        assertEquals("UsuarioNuevo", u.getName());
    }
}
```

```

@Test
@DisplayName("Given existing email, When creating user, Then error is thrown")
void duplicatedEmailBDD() {
    userService.createUser(1L, "Existente", "dup@example.com");

    IllegalStateException ex = assertThrows(
        IllegalStateException.class,
        () -> userService.createUser(2L, "Otro", "dup@example.com")
    );

    assertEquals("Ya existe un usuario con ese email", ex.getMessage());
}
}

```

## 8. Tabla Comparativa Final

| Tipo de prueba          | Qué valida                  | Dependencias           | Velocidad  | Enfoque              |
|-------------------------|-----------------------------|------------------------|------------|----------------------|
| <b>Unitarias</b>        | Lógica interna del servicio | Mock del repositorio   | Muy alta   | Reglas de negocio    |
| <b>Integración</b>      | Servicio + repositorio real | InMemoryUserRepository | Media      | Flujo completo       |
| <b>Aceptación (BDD)</b> | Comportamiento esperado     | Servicio + repositorio | Media-baja | Historias de usuario |

## 9. Actividades del Taller

1. Clonar el proyecto o copiar el código en un nuevo proyecto Maven.
2. Ejecutar las pruebas con:

```
mvn test
```

3. Agregar una nueva regla de negocio (ejemplo: validar formato del email).
4. Crear **dos pruebas unitarias** para esa regla.
5. Crear **una prueba de integración** que valide el flujo completo.
6. Crear **un escenario BDD** que represente un criterio de aceptación.